

Documentación completa: CV PPTX API — Flujo y arquitectura

Proyecto: cv-pptx-api (Sofijobs automatización)

Versión: 1.0.0

Última actualización: Marzo 2026

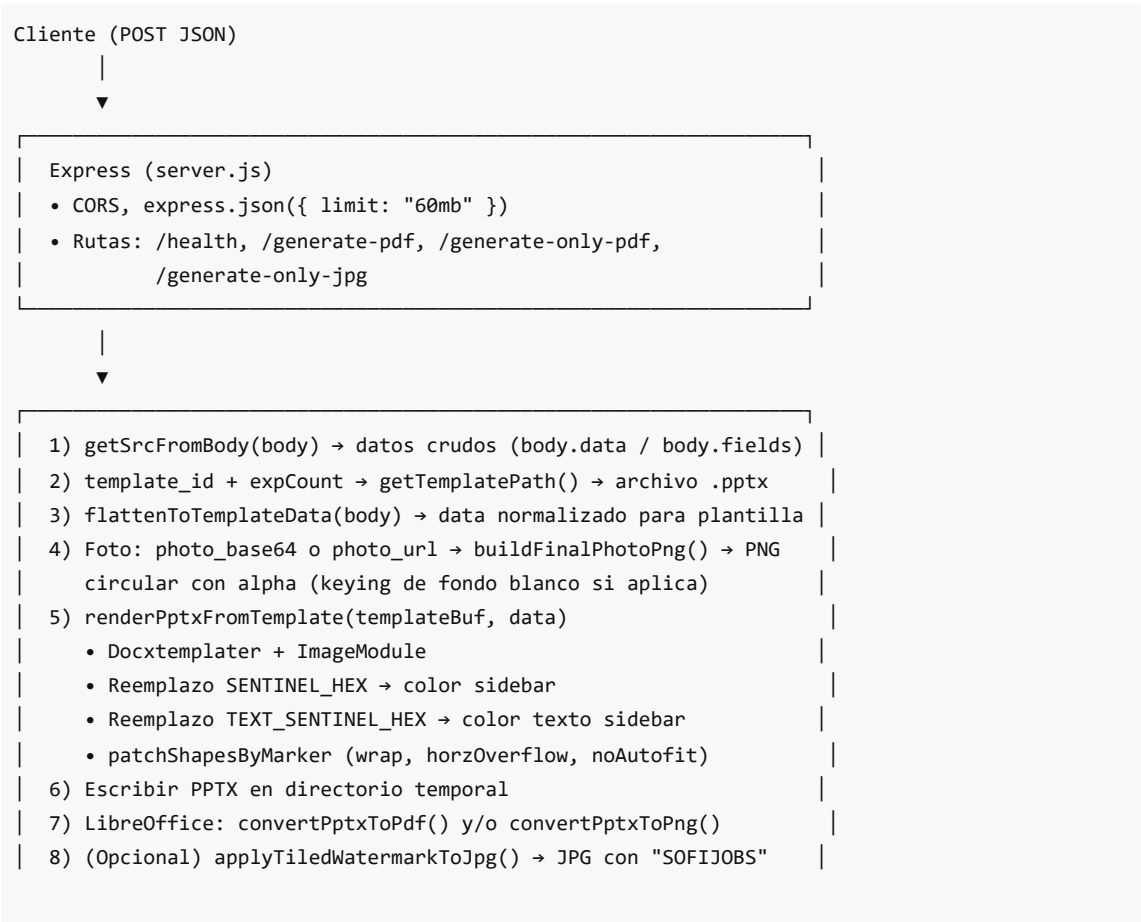
1. Introducción y propósito

La **CV PPTX API** es un servicio backend que:

- Recibe datos de currículum (nombre, título, experiencia, educación, contacto, foto, color de acento, etc.).
- Elige una **plantilla PPTX** según `template_id` y la cantidad de experiencias (variantes v1–v7).
- Rellena la plantilla con **Docxtemplater** usando placeholders `{{name}}`, `{{title}}`, `{{%photo}}`, etc.
- Aplica **color de acento** (reemplazo de colores sentinelas en todos los XML del PPTX).
- Procesa la **foto** (recorte circular, eliminación de fondo blanco si aplica).
- Genera **PDF** (vía LibreOffice) y opcionalmente **JPG** con marca de agua.
- Expone endpoints para descargar: **ZIP (PDF + JPG)**, **solo PDF** o **solo JPG**.

Todo el flujo está pensado para integrarse con formularios o sistemas que envían un JSON con los datos del CV.

2. Arquitectura general



9) Respuesta: ZIP (PDF+JPG), o solo PDF, o solo JPG

- **Entrada:** JSON en el body (o anidado en `body.data` / `body.fields`).
- **Salida:** archivo binario (ZIP, PDF o JPG) con `Content-Disposition: attachment` .

3. Dependencias (package.json)

Paquete	Uso
express	Servidor HTTP y rutas.
cors	Habilitar CORS para llamadas desde frontend.
sharp	Procesamiento de imagen: redimensionar, alpha, keying de fondo, máscara circular.
archiver	Crear ZIP en memoria para <code>/generate-pdf</code> (PDF + JPG).
pizzip	Leer/escribir PPTX (ZIP con XML).
docxtemplater	Sustituir placeholders en PPTX (XML interno).
docxtemplater-image-module-free	Inserción de imágenes (foto, iconos) en plantilla.
node-fetch	No usado directamente en <code>server.js</code> ; puede usarse en otros módulos.
pptxgenjs	Declarado; la generación real se hace con plantillas + Docxtemplater.

En producción el Dockerfile ejecuta `npm ci --omit=dev` , por lo que **todas** las dependencias de runtime deben estar en `dependencies` (incluido `archiver`).

4. Variables de entorno

Variable	Descripción	Valor por defecto
PORT	Puerto HTTP del servidor.	3000
DEFAULT_TEMPLATE_ID	Plantilla por defecto si no se envía <code>template_id</code> .	"1"
SOFFICE_PATH	Ruta al ejecutable de LibreOffice (<code>soffice</code>).	Windows: <code>C:\Program Files\LibreOffice\program\soffice.exe</code> ; Linux: <code>soffice</code>
SENTINEL_HEX	Color sentinela en el PPTX para fondo del sidebar/acentos. Se reemplaza por el color elegido.	c0504d

TEXT_SENTINEL_HEX	Color sentinela para texto del sidebar (se reemplaza por blanco o negro según contraste).	543F3F
DEBUG_COLOR	Si "1", se loguean los archivos XML tocados en el reemplazo de color.	(vacío)

5. Estructura del proyecto

```

cv-pptx-api/
├─ server.js           # Punto de entrada; toda la lógica de API y flujo
├─ package.json
├─ package-lock.json
├─ dockerfile         # Imagen Node 20 + LibreOffice + npm ci
├─ templates/         # Plantillas PPTX (Plantilla_oficial_*_v*.pptx)
├─ assets/
│   └─ icons/         # Iconos sidebar: light/ y dark/ (phone.png, mail.png, etc.)
├─ docs/
│   └─ DOCUMENTACION_FLUJO_CV_PPTX_API.md / .pdf
└─ (opcional) .dockerignore, .gitignore, README.md

```

Las plantillas se nombran según `TEMPLATE_VARIANTS` en `server.js` (ver sección 8).

6. Endpoints de la API

6.1 GET `/health`

- **Respuesta:** `{ "ok": true } .`
- Uso: comprobar que el servicio está vivo (Render, load balancers, etc.).

6.2 POST `/generate-pdf`

- **Body:** JSON con datos del CV (ver sección 9). Acepta hasta **60 MB** (para foto en base64).
- **Respuesta:** archivo ZIP (`application/zip`) con:
 - `{nombreSeguro}.pdf` — CV en PDF.
 - `{nombreSeguro}.jpg` — Primera página como JPG con marca de agua "SOFIJOBS".
- **Nombre del archivo ZIP:** `{nombreSeguro}_pack.zip` (nombre derivado de `name / nombre` del body, sanitizado).

6.3 POST `/generate-only-pdf`

- **Body:** mismo JSON que `/generate-pdf` .
- **Respuesta:** solo el PDF (`application/pdf`), nombre `{nombreSeguro}.pdf` .

6.4 POST `/generate-only-jpg`

- **Body:** mismo JSON que `/generate-pdf` .
- **Respuesta:** solo el JPG con marca de agua (`image/jpeg`), nombre `{nombreSeguro}.jpg` .

En todos los endpoints POST, si ocurre un error se devuelve `500` con cuerpo JSON: `{ error: string, stack: string }`.

7. Flujo detallado paso a paso

1. Recepción del body

Se toma `req.body`. Si viene `body.data` o `body.fields` (objeto), se usa como fuente de datos; si no, se usa `body` directamente (`getSrcFromBody`).

2. Identificación de plantilla y variante

- `template_id` : de body o de `src` (por defecto `DEFAULT_TEMPLATE_ID`, ej. 1).
- Se cuenta la cantidad de experiencias con `countExperiencesFromSrc(src, 7)` (máximo 7).
- Se considera la preferencia de "CV en 2 páginas" (`getTwoPagesAnswerFromSrc` → `normalizeTwoPagesAnswer`).
- Si la respuesta es "No, por favor que sea de una página" o "Sólo si lo consideran estrictamente necesario..." → se fuerza variante de 1 página (equivalente a cap 5 experiencias).
- Si es "Sí, prefiero que esté bien desarrollado..." y el template permite 2 páginas → se usa hasta 6 o 7 experiencias.
- `capExpCountForTemplate(templateId, expCount)` limita según si el template es de 2 páginas (9, 11, 12, 13) o no.
- Con `template_id` y `expCountForVariant` se resuelve el archivo .pptx con `getTemplatePath(templateId, expCountForVariant)` (ver sección 8).

3. Datos para la plantilla

`flattenToTemplateData(body)` construye el objeto `data` que Docxtemplater usará: normaliza nombres de campos, aplica límites de caracteres, formatea educación, experiencia, idiomas, IT, cursos, sidebar, contacto, etc. (ver sección 9).

4. Foto

- Si hay `photo_base64` (o alias): se decodifica con `decodeBase64Image`.
- Si hay `photo_url` : se descarga con `fetchBufferFromUrl` (soporta Google Drive con normalización de URL).
- Con el buffer se llama a `buildFinalPhotoPng(photoBuf, { W, H })` (tamaño por `TEMPLATE_PROFILES[template_id].photoSize`): recorte, detección de fondo claro, keying a alpha, redimensionado y máscara circular.
- El resultado se asigna a `data.photo` en formato `data:image/png;base64,...` para el `ImageModule`.

5. Render del PPTX

- Se lee el archivo de plantilla desde disco.
- `renderPptxFromTemplate(templateBuf, data)` :
 - Calcula color de acento con `resolveAccentHex(data.accent_color_raw)` y color de texto del sidebar (`pickTextColorForSidebar`).
 - Configura `ImageModule` (`getImage`, `getSize`) para la etiqueta `photo` y para iconos.
 - Docxtemplater rellena todos los placeholders.
 - `replaceColorInAllXml` reemplaza `SENTINEL_HEX` por el color del sidebar y `TEXT_SENTINEL_HEX` por el color de texto.
 - Se aplican parches de sidebar (`patchShapesByMarker` con `__SB_TITLE__` y `__SB_BODY__` : `wrap`, `horzOverflow clip`, `noAutofit`).

- Se compacta espaciado en shapex de educación y se eliminan párrafos con viñetas vacíos.
- El buffer PPTX resultante se escribe en un directorio temporal.

6. Conversión a PDF y PNG

- `convertPptxToPdf(pptxPath, tmpDir)` : ejecuta LibreOffice en headless (`--convert-to pdf`).
- Para ZIP o JPG: `convertPptxToPng(pptxPath, tmpDir)` genera PNG(s); se toma el primero ordenado.

7. JPG con marca de agua

- `applyTiledWatermarkToJpg(pngPath, jpgOut, { text: "SOFIJOBS", ... })` : usando Sharp y un SVG con patrón de texto, se genera el JPG final.

8. Respuesta

- `/generate-pdf` : se usa `archiver` para crear un ZIP en memoria con el PDF y el JPG y se envía con `Content-Disposition: attachment; filename="..._pack.zip"` .
- `/generate-only-pdf` : se envía el buffer PDF.
- `/generate-only-jpg` : se envía el buffer JPG.

Los archivos temporales quedan en el sistema de archivos del servidor (en producción suele limpiarse al terminar la request o por el SO).

8. Sistema de plantillas (template_id y variantes v1–v7)

8.1 Template ID

- template_id** válido: **1 a 14**.
- Se puede enviar como `template_id` , `template` o `templateId` en el body o dentro de `data / fields` .
- Si no se envía, se usa `DEFAULT_TEMPLATE_ID` (por defecto `"1"`).

8.2 Variantes por cantidad de experiencias

Cada `template_id` tiene hasta 7 variantes de archivo PPTX (v1–v7), según cuántas experiencias tenga el CV:

Experiencias	Variante	Uso típico
0–1	v1	Poca experiencia
2	v2	2 experiencias
3	v3	3 experiencias (base)
4	v4	4 experiencias
5	v5	5 experiencias (1 página)
6	v6	6 experiencias (2 páginas)
7+	v7	7 experiencias (2 páginas)

- La función `wantedVariantKeyByExpCount(expCount)` devuelve la clave (v1–v7).

- `pickVariantFileName(templateId, expCount)` busca en `TEMPLATE_VARIANTS[templateId]` el archivo correspondiente; si no existe esa variante, prueba otras en orden de cercanía y finalmente fallback a v3 o al primer archivo existente.
- **Plantillas de 2 páginas:** solo los **template_id 9, 11, 12 y 13** pueden usar v6/v7 (6–7 experiencias). Para el resto, `capExpCountForTemplate` limita a 5 experiencias y por tanto a variante v5 como máximo.

8.3 Preferencia “CV en 2 páginas”

Si el formulario incluye una pregunta tipo “¿Te gustaría que tu CV tenga dos páginas?”:

- “**No, por favor que sea de una página**” → `one_page` → se fuerza a 5 experiencias (v5).
- “**Sólo si lo consideran estrictamente necesario...**” → `only_if_needed` → también se fuerza v5.
- “**Sí, prefiero que esté bien desarrollado toda mi experiencia**” → `prefer_two_pages` → se permite v6/v7 cuando el template sea 9, 11, 12 o 13 y haya 6 o 7 experiencias.

La búsqueda del valor se hace por keys como `two_pages`, `dos_paginas`, etc., o por texto de la pregunta en el nombre del campo (`getTwoPagesAnswerFromSrc`).

8.4 Nombres de archivos de plantilla

Patrón: `Plantilla_oficial_{id}_v{v}.pptx` o `Plantilla_oficial_{id}.pptx` (v3).

Para `template_id 1` se usa el nombre base “Plantilla_oficial_1_verde”.

Ejemplo para id 11: `Plantilla_oficial_11.pptx` (v3), `Plantilla_oficial_11_v1.pptx`, ..., `Plantilla_oficial_11_v7.pptx`.

9. Datos de entrada (body) y campos para la plantilla

9.1 Estructura del body

El servidor acepta:

- **body** con campos en la raíz, o
- **body.data** (objeto), o
- **body.fields** (objeto).

Además se puede enviar en la raíz del body:

- `template_id` (o `template`)
- `accent_color_raw` (también puede ir dentro de `data/fields`)

El resto de campos pueden estar en `body`, `body.data` o `body.fields`. La función `getSrcFromBody(body)` devuelve ese objeto “fuente” y `flattenToTemplateData(body)` construye el objeto `data` que usa Docxtemplater.

9.2 Listado de campos (`flattenToTemplateData`)

A continuación se listan los campos que se leen y cómo se exponen en `data` (nombres que deben coincidir con los placeholders del PPTX). Los alias entre paréntesis son los que se buscan en la fuente si no existe el nombre principal.

Identificación y preferencias

- `template_id` (`template`, `templateId`)
- `two_pages_answer_raw` / `two_pages_pref` (derivados de la pregunta de 2 páginas)

Foto y color

- `photo_base64` (`photoBase64`, `photo`)
- `photo_url` (`photoUrl`)
- `accent_color_raw` (`colores_raw`, `colors_raw`, `colores`, `colors`)

Cabecera

- `name` (`nombre`)
- `title` (`título`)
- `about` (`objective`, `objetivo`)

Contacto

- `contact_phone` (`phone`, `telefono`)
- `contact_email` (`email`)
- `contact_location` (`location`, `ubicacion`)
- `contact_website` (`website`, `web`)
- `licencia` (`licencia_conducir`, `driver_license`)

Experiencia (1–7)

Para cada `n` de 1 a 7:

- `exp_n_dates`
- `exp_n_company`
- `exp_n_role`
- `exp_n_b1` ... `exp_n_b5` (`bullets`)
- `exp_n_bullets_block` (generado: concatenación de `bullets` con "• ")

Educación (1–3)

- `edu_1_school` , `edu_1_degree` , `edu_1_years`
- `edu_2_school` , `edu_2_degree` , `edu_2_years`
- `edu_3_school` , `edu_3_degree` , `edu_3_years`

Idiomas, IT, Cursos

- `idiomas_raw` (`idiomas`, `languages_raw`) — se divide y se asigna a `idioma_1` , `idioma_2` , `idioma_3`
- `it_raw` (`it`, `informatica_raw`) — se divide y se asigna a `it_1` ... `it_6`
- `cursos_raw` (`cursos`, `courses_raw`) — se divide y se asigna a `curso_1` ... `curso_6`
- O bien directamente: `idioma_1` , `idioma_2` , `idioma_3` , `it_1` ... `it_6` , `curso_1` ... `curso_6`

Competencias

- `skill_1` ... `skill_7`

Sidebar (generados)

- `sidebar_sections` — secciones completas (Educación, Cursos, Informática, Idiomas, Competencias)
- `sidebar_sections_noedu` — sin educación
- `sidebar_sections_eduycursos` — solo formación académica y cursos
- `sidebar_sections_full` — incluye contacto como sección
- `sidebar_contact` — bloque contacto (iconos + texto)
- `contact_rows` — filas de contacto con iconos (para plantillas que usan `ImageModule` en iconos)

Internos (color)

- `_sidebarHex` , `_sidebarTextHex` — calculados a partir de `accent_color_raw` ; no son placeholders pero se usan en reemplazo de color.

9.3 Límites de caracteres (LIMITS)

Los valores se recortan (si `LIMITS.ENABLE_CLAMP === true`) según:

- name: 200 | title: 240 | about: 6000 (o el max del perfil de la plantilla)
- contact: email 180, phone 80, location 240, website 220
- experiencia: role 400, company 400, dates 120, cada bullet 220
- educación: school 380, degree 380, years 120
- skill 200 | item (idiomas/it/cursos) 260

Por defecto `ENABLE_CLAMP` está en `false` , es decir no se recorta por código.

9.4 Normalizaciones aplicadas

- **Texto general:** `safeStr` , normalización NFC, “undefined”/“null” → vacío.
- **Nombre de instituciones:** diccionario `CANONICAL_INSTITUTIONS` (UBA, UNLP, UTN, etc.) y `canonicalizeInstitutionRobust` , `normalizeInstitutionField` .
- **Experiencia:** `normalizeTechNames` (Power BI, Excel, JavaScript, etc.), `replaceForbiddenWords` , `stripPipesFromBullets` , `segmentBulletsIfNeeded` , `removeDuplicateBullets` , `normalizeExperienceVisualGroup` .
- **Educación:** formato "DEGREE | SCHOOL (YEARS)", `normalizeEducationVisualGroup` , `normalizeStatusWords` (Finalizado, En curso).
- **Idiomas:** `enforceLevelFormat(..., "idioma")` → formato "Idioma | nivel X", filtro `shouldDropLanguageItem` (ej. español nativo se descarta).
- **IT:** `enforceLevelFormat(..., "it")` , `normalizeITNoParens` , colapso de “Microsoft Office” a un solo ítem con nivel promedio.
- **Cursos:** `normalizeCourseLine` (título | lugar (fechas)), `normalizeRomanNumerals` , `normalizeTechNames` .
- **Teléfono:** `normalizePhone` (código país por defecto 54, formato +54...).
- **Color:** `resolveAccentHex` (hex o palabras de color en español, mapa `COLOR_MAP`, suavizado si es muy vivo).

10. Placeholders en la plantilla PPTX

En el PPTX se usan delimitadores `{{ y }}` (Docxtemplater).

Texto:

- `{{name}}` , `{{title}}` , `{{about}}`
- `{{contact_phone}}` , `{{contact_email}}` , `{{contact_location}}` , `{{contact_website}}`
- `{{exp_1_role}}` , `{{exp_1_company}}` , `{{exp_1_dates}}` , `{{exp_1_b1}}` ... `{{exp_1_b5}}` , `{{exp_1_bullets_block}}` (y lo mismo para exp_2 ... exp_7)
- `{{edu_1_degree}}` , `{{edu_1_school}}` , `{{edu_1_years}}` (y edu_2, edu_3)
- `{{skill_1}}` ... `{{skill_7}}`
- `{{idioma_1}}` , `{{idioma_2}}` , `{{idioma_3}}`
- `{{it_1}}` ... `{{it_6}}`
- `{{curso_1}}` ... `{{curso_6}}`

Imagen (foto):

- `{{%photo}}` — placeholder de imagen; el valor debe ser Buffer o string base64 (en `data.photo` se pasa como `data:image/png;base64,...` y el ImageModule lo decodifica). El tamaño lo define `TEMPLATE_PROFILES[template_id].photoSize` (ej. [520, 520]).

Iconos (ImageModule):

- Si la plantilla usa etiquetas tipo `icon` o que terminen en `_icon`, se pueden usar con `contact_rows` (cada fila con `icon + text`); el servidor rellena iconos desde `assets/icons/light/` o `dark/` según el color de texto del sidebar.

Sidebar por secciones:

- Las secciones se construyen como listas de objetos con `title`, `line` (subrayado), `body`. Dependiendo del diseño del PPTX pueden usarse bloques como `sidebar_sections`, `sidebar_contact`, etc., con loops en Docxtemplater.

Marcadores para parche de sidebar:

- En el XML del PPTX, shapes que contengan el texto **SB_TITLE** o **SB_BODY** se parchean (wrap, horzOverflow, noAutofit); después ese texto se borra para no verse en la diapositiva.

11. Procesamiento de la foto

1. Origen:

- `photo_base64` : string (con o sin prefijo `data:image/...;base64,`).
- `photo_url` : URL de imagen; si es Google Drive se normaliza a `https://drive.google.com/uc?export=download&id=...` .

2. Descarga (solo URL):

- `fetchBufferFromUrl` : sigue redirecciones, User-Agent tipo navegador.

3. `buildFinalPhotoPng(photoBuf, { W, H })`:

- Rotar según EXIF (`sharp(photoBuf).rotate()`).
- Recorte de bordes (`trim`), resize a W×H con `fit: "cover"` , centro.
- Asegurar canal alpha (`ensureAlpha`), salida PNG.
- Muestreo de color de esquinas (`sampleCornerColor`); si el fondo es claro (promedio ≥ 210), se aplica **keying**: `keyOutBackgroundToAlpha` con umbral y suavidad para convertir fondo blanco/similar en transparente.
- Redimensionado final y luego **forceCircleTransparentOutside**: máscara SVG circular (centro, radio según tamaño), `blend dest-in` sobre la imagen → fuera del círculo queda transparente.
- Resultado: PNG circular con transparencia, sin aplanar la imagen final.

4. Inserción en plantilla:

- Se asigna a `data.photo` como `data:image/png;base64,...` .
- El ImageModule de Docxtemplater reconoce la etiqueta `photo` (o `%photo`) y devuelve el buffer y el tamaño [W, H] según el perfil de la plantilla.

12. Sistema de colores

12.1 Colores sentinela en el PPTX

- En LibreOffice (o el editor donde se diseñe la plantilla) se debe usar **exactamente**:
 - **SENTINEL_HEX** (por defecto `c0504d`) para: fondo del sidebar, barras, acentos, líneas.
 - **TEXT_SENTINEL_HEX** (por defecto `543F3F`) para: todo el texto del sidebar que deba tener contraste automático (blanco o negro).

El servidor reemplaza en **todos** los XML del PPTX:

- Cada aparición de `SENTINEL_HEX` → color de sidebar (o gris por defecto si no hay color).
- Cada aparición de `TEXT_SENTINEL_HEX` → color de texto del sidebar (blanco o negro según luminancia).

12.2 Cálculo del color de acento

- **resolveAccentHex(accent_color_raw)**:
 - Si el usuario pide "sin color" / "blanco y negro" / "ATS" / "Harvard" / etc. → no se aplica color (string vacío).
 - Si se pasa un hex de 6 caracteres (# opcional) → se usa (y se suaviza si `isTooVibrantHex`).
 - Si se pasa una palabra (ej. "azul", "verde", "rosa claro") → se busca en `COLOR_MAP` y se devuelve el hex; si no hay coincidencia se usan heurísticas (ej. "azul" → tono azul por defecto).
- **pickSidebarColorForWhiteText(accentHex)**: si el color es muy claro (luminancia > 0.6), se oscurece para que el texto blanco sea legible.
- **pickTextColorForSidebar(sidebarHex, accentRaw)**: por luminancia se elige negro o blanco; para algunos colores (naranja, ocre, marrón) se fuerza blanco.

12.3 Iconos del sidebar

- Si el color de texto del sidebar es blanco → tema **light** (iconos claros).
- Si es negro → tema **dark**.
- Los iconos se cargan desde `assets/icons/{light|dark}/` (phone.png, mail.png, location.png, car.png) y se cachean en memoria (`loadIconBuffer`).

13. Sidebar: parches XML y overflow

Para evitar que el texto del sidebar se desborde o se recorte mal en LibreOffice:

- Se buscan en slides, slideLayouts y slideMasters los shapes que contienen **SB_TITLE** o **SB_BODY**.
- En cada shape se reemplaza el marcador por texto vacío (para que no se vea) y se aplica **patchBodyPr**:
 - `wrap="square"`
 - `horzOverflow="clip"` (corta el overflow horizontal)
 - `vertOverflow="overflow"` (opcional)
 - Se eliminan hijos de autofit y se fuerza `<a:noAutofit/>` para no redimensionar la caja.

Además se puede compactar el espaciado entre párrafos en shapes que contienen placeholders de educación (`edu_1_degree` , etc.) con `forceCompactParagraphSpacingInShape` .

Y se eliminan párrafos con viñeta que queden vacíos (`removeEmptyBulletedParagraphs`).

14. LibreOffice: PPTX → PDF y PPTX → PNG

- **SOFFICE_PATH**: en Windows suele ser la ruta a `soffice.exe` ; en Linux (Docker/Render) es `soffice` (instalado con `apt-get install libreoffice libreoffice-impress ...`).
- **convertPptxToPdf(pptxPath, outDir)**:

- Argumentos: `--headless` , `--nologo` , `--nofirststartwizard` , `--norestore` , `--convert-to pdf` , `--outdir` , `outDir` , `pptxPath` .
 - El PDF generado tiene el mismo nombre que el PPTX con extensión `.pdf` en `outDir` .
 - **convertPptxToPng(pptxPath, outDir):**
 - Mismo esquema con `--convert-to png` .
 - LibreOffice genera uno o varios PNG (una por diapositiva); el código toma el primero ordenado alfabéticamente (normalmente la primera diapositiva).
-

15. Marca de agua en JPG

applyTiledWatermarkToJpg(inputPngPath, outputJpgPath, options):

- Lee el PNG generado por LibreOffice.
 - Crea un SVG con un **patrón** de texto (ej. "SOFIJOBS") repetido en cuadrícula, con ángulo `-28°`, opacidad y color configurables.
 - Compone el patrón sobre la imagen con Sharp y exporta a JPG (calidad 92, mozjpeg).
 - Parámetros por defecto: `text: "SOFIJOBS"` , `color: "#9CA3AF"` , `opacity: 0.22` , `fontSize: 44` , `angle: -28` , `stepX: 420` , `stepY: 240` .
-

16. Generación del ZIP (/generate-pdf)

- Se crea un directorio temporal, se genera el PPTX, se convierte a PDF y a PNG, se aplica la marca de agua al JPG.
 - Con **archiver** se crea un ZIP en memoria:
 - Se añade el buffer del PDF con nombre `{fileBase}.pdf` .
 - Se añade el archivo JPG con nombre `{fileBase}.jpg` .
 - Se hace `archive.pipe(res)` y se envían los headers:
 - `Content-Type: application/zip`
 - `Content-Disposition: attachment; filename="{fileBase}_pack.zip"`
 - `fileBase` viene de `toSafeFilename(name)` (nombre del candidato sanitizado) o "CV" si no hay nombre.
-

17. Despliegue (Docker y Render)

17.1 Dockerfile

- Base: **node:20-bookworm**.
- Instalación de LibreOffice: `libreoffice` , `libreoffice-impress` , fuentes (DejaVu, Liberation, Noto, etc.), `fontconfig` .
- `WORKDIR /app` .
- `COPY package.json package-lock.json ./` → `RUN npm ci --omit=dev` y comprobación de que `archiver` está instalado.
- `COPY . .` (resto del proyecto).
- `ENV PORT=3000` , `EXPOSE 3000` , `CMD ["node", "server.js"]` .

Importante: en el build no se deben omitir dependencias de producción; `archiver` debe estar en `dependencies` de `package.json` .

17.2 Render (u otro host)

- El servicio debe exponer el **PORT** que asigne la plataforma (Render inyecta PORT).
 - Si se usa Docker, la imagen se construye con el Dockerfile anterior; si se usa “Native Environment” de Render, el build suele ser `npm install + node server.js`; en ese caso también hace falta que `package.json` incluya todas las dependencias.
 - Para evitar caché de build viejo, en Render se puede usar “Clear build cache” y volver a desplegar tras cambios en `package.json` o en el Dockerfile.
-

18. Resumen de constantes y configuración relevante

- **LIMITS.ENABLE_CLAMP:** false → no recorte por longitud en campos.
 - **TEMPLATE_PROFILES:** por `template_id`, `about.maxChars` y `photoSize` [ancho, alto].
 - **TWO_PAGE_TEMPLATES:** Set { 9, 11, 12, 13 }.
 - **TEMPLATE_VARIANTS:** objeto `template_id` → { v1 ... v7 } con nombres de archivo.
 - **TEMPLATE_MAP:** `template_id` → nombre del archivo base (v3).
 - **COLOR_MAP:** palabras en español → hex (violeta, rosa, azul, verde, gris, etc.).
 - **CANONICAL_INSTITUTIONS:** nombres de universidades/instituciones → siglas (UBA, UNLP, UTN, etc.).
 - **Delimitadores Docxtemplater:** { { y } } .
 - **Límite body:** 60 MB.
-

19. Diagrama de flujo simplificado (texto)

```
POST /generate-pdf (body JSON)
  → getSrcFromBody
  → template_id, countExperiences, twoPagesPreference
  → getTemplatePath(templateId, expCountForVariant) → .pptx
  → flattenToTemplateData(body) → data
  → photo: base64/url → buildFinalPhotoPng → data.photo
  → renderPptxFromTemplate(templateBuf, data)
    → replaceColorInAllXml (SENTINEL, TEXT_SENTINEL)
    → patchShapesByMarker (sidebar)
  → write PPTX to tmp
  → convertPptxToPdf, convertPptxToPng
  → applyTiledWatermarkToJpg → JPG
  → archiver ZIP (PDF + JPG) → response
```

Documento generado a partir del código de **server.js** del proyecto **cv-pptx-api**. Para detalles exactos de constantes o nombres de archivo, consultar el fuente.